

QUIZ 2B/BB

Glenn Kohler

3 3/4 Tte

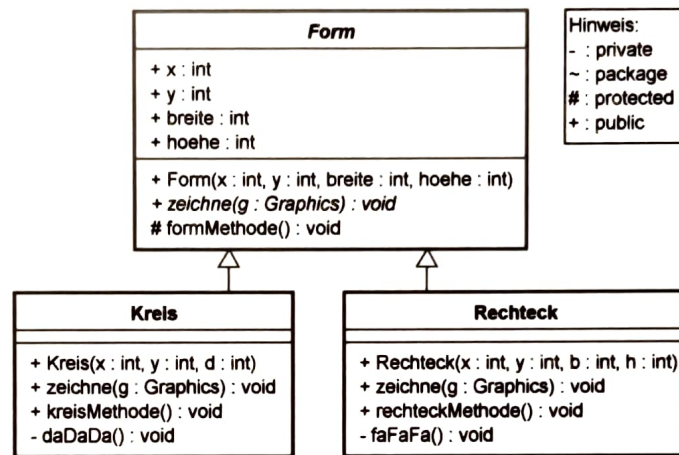
Bedingungen:

- Die Aufgaben sind ohne jegliche Unterlagen direkt in wenigen Worten auf der Aufgabenstellung oder allenfalls auf einem Zusatzblatt zu beantworten.

1) Wir haben im Unterricht nachfolgende Struktur, die nun noch ergänzt wurde, näher angeschaut:

&

2)



Hinweis:
- : private
~ : package
: protected
+ : public

Mittels der Zeile

```
public Form form = new Kreis(40, 50, 30);
```

werde ein Objekt der Art Kreis erzeugt. Welche Methoden und Attribute sind mittels **form.** im gleichen Package sichtbar?

form, form, form.zeichne und form.formMethode

x, y, breite, hoehe, zeichne() -> Methode von form, formMethode()
Kreis

Die Form **form** werde nun in einen Kreis gewandelt:

```
public Kreis kreis = (Kreis)form;
```

Welche Methoden und Attribute sind mittels **kreis.** im gleichen Package sichtbar?

form, zeichne, kreisMethode und daDaDa

x, y, breite, hoehe, zeichne (von Kreis), kreisMethode, form und formMethode

↑
konstruktoren NICHT sichtbar

- 3) Was ist unter Polymorphismus zu verstehen? Verwenden Sie zur Erläuterung das Bsp. in Aufgabe 1).

Unter Polymorphismus versteht man die Untergliederung von mehreren Klassen:

Hier im BSP: Kreis und Rechteck werden Polymorph der Klasse Form zugeordnet zugeordnet.
Kreis und Rechteck kann also in vielerlei Gestalt auftreten (versch. Grössen)
sind aber immer Formen mit x, y, breite und höhe.

Als Beilage finden Sie den Code des Taschenrechners. Beantworten Sie die nachfolgenden Fragen anhand des Codes.

- 4) Welche Methode ist durch das Interface ActionListener sichtbar, welche durch das Interface Observer?

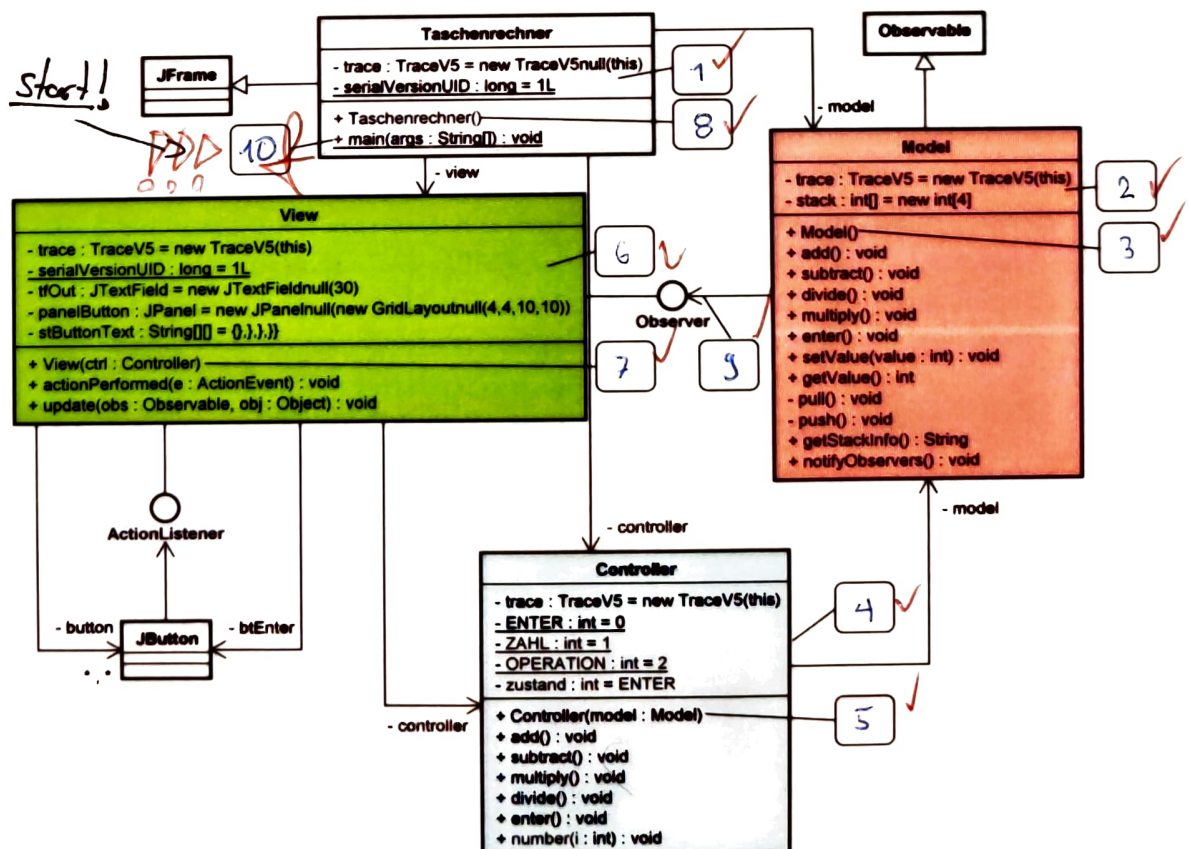
Durch ActionListener sichtbar

Durch Observer sichtbar

- actionPerformed ✓

- update ✓

- 5) Beim Aufstarten hinterlässt das Programm eine Spur. Nummerieren Sie die Kästchen in der richtigen Reihenfolge. Tipp: Code im Anhang beachten!



3/4

- 6) Der Controller befinde sich im Zustand ENTER. Auf die Eingabeabfolge **7, 7, ENTER, ***, hin, hinterlässt das Programm eine Spur. Nummerieren Sie die Zeilen in der richtigen Reihenfolge. Die Lösung ist in Weiss eingetragen ;-) ...
- 7)
- &
- 8)

_____	Methode Model@172292469.getValue() wird ausgeführt ...
_____	Methode Controller@237919564.multiply() wird ausgeführt ...
<u>5</u> ✓	Methode View@1219707368.update() wird ausgeführt ...
_____	Methode Model@172292469.getValue() wird ausgeführt ...
<u>0</u>	Methode View@1219707368.actionPerformed() wird durch Ereignis ausgelöst ...
_____	Methode Controller@237919564.enter() wird ausgeführt ...
_____	Methode View@1219707368.update() wird ausgeführt ...
_____	Methode Model@172292469.enter() wird ausgeführt ...
_____	Methode Model@172292469.notifyObservers() wird ausgeführt ...
_____	Methode Model@172292469.multiply() wird ausgeführt ...
_____	Methode Model@172292469.getValue() wird ausgeführt ...
_____	Methode Model@172292469.setValue() wird ausgeführt ...
<u>1</u> ✓	Methode Controller@237919564.number() wird ausgeführt ...
_____	Methode View@1219707368.update() wird ausgeführt ...
_____	Methode Model@172292469.getValue() wird ausgeführt ...
_____	Methode Model@172292469.notifyObservers() wird ausgeführt ...
<u>10</u>	Methode View@1219707368.actionPerformed() wird durch Ereignis ausgelöst ...
_____	Methode Model@172292469.notifyObservers() wird ausgeführt ...
<u>4</u> ✓	Methode Model@172292469.notifyObservers() wird ausgeführt ...
<u>2</u>	Methode Model@172292469.getValue() wird ausgeführt ...
_____	Methode Model@172292469.pull() wird ausgeführt ...
<u>20</u>	Methode View@1219707368.actionPerformed() wird durch Ereignis ausgelöst ...
_____	Methode Controller@237919564.number() wird ausgeführt ...
<u>30</u>	Methode View@1219707368.actionPerformed() wird durch Ereignis ausgelöst ...
_____	Methode Model@172292469.push() wird ausgeführt ...
_____	Methode View@1219707368.update() wird ausgeführt ...
<u>3</u> ✓	Methode Model@172292469.setValue() wird ausgeführt ...

P.S.: Bitte Bleistift verwenden und so lange radieren bis es passt ;-)!

Taschenrechner

```
public class Taschenrechner extends JFrame {
    private TraceVS trace = new TraceVS(this);
    private static final long serialVersionUID = 1L;
    private Model model = new Model();
    private Controller controller = new Controller(model);
    private View view = new View(controller);

    public Taschenrechner() {
        trace.constructorCall();
        model.addObserver(model, view);
        getContentPane().add(view);
    }

    public static void main(String args[]) {
        TraceVS.mainCall(true, true, true);
        SwingUtilities.invokeLater(new Runnable() {
            public void run() {
                Taschenrechner frame = new Taschenrechner();
                frame.setDefaultCloseOperation(JFrame.EXIT_ON_CLOSE);
                frame.pack();
                frame.setMinimumSize(frame.getPreferredSize());
                frame.setVisible(true);
            }
        });
    }
}
```

View

```
public class View extends JPanel implements Observer, ActionListener {
    TraceVS trace = new TraceVS(this);
    private JButton bEnter = new JButton("Enter");
    private JPanel panelButton = new JPanel(new GridLayout(4, 4, 10, 10));
    private JButton[][] button = new JButton[4][4];
    private String[][] buttonText = { { "7", "8", "9", "/" }, { "4", "5", "6", "*" },
                                      { "1", "2", "3", "-" }, { "0", ".", "+", "=" } };

    private Controller controller;

    public View(Controller ctrl) {
        super(new BorderLayout(10, 10));
        trace.constructorCall();
        this.controller = ctrl;
        for (int i = 0; i < button.length; i++) {
            for (int k = 0; k < button[0].length; k++) {
                button[i][k] = new JButton(buttonText[i][k]);
                panelButton.add(button[i][k]);
                button[i][k].addActionListener(this);
            }
        }
        add(this, BorderLayout.NORTH);
        add(panelButton, BorderLayout.CENTER);
        add(bEnter, BorderLayout.SOUTH);
        bEnter.addActionListener(this);
    }

    public void actionPerformed(ActionEvent e) {
        trace.eventCall();
        switch (((JButton) e.getSource()).getText()) {
            case "+/-":
                controller.divide();
                break;
            case "+":
                controller.multiply();
                break;
            case "-":
                controller.add();
                break;
            case "*":
                controller.subtract();
                break;
            case "Enter":
                controller.enter();
                break;
            case "=":
                break;
            default:
                int zahl1 = Integer.parseInt(((JButton) e.getSource()).getText());
                controller.number(zahl1);
        }
    }
}
```

InputPanel

```
public class InputPanel extends JPanel implements ActionListener {
    private static final long serialVersionUID = 1L;
    private TraceVS trace = new TraceVS(this);
    public JTextField tfAnzahlPunkte = new JTextField("0.0");
    public JTextField tfMaxPunkte = new JTextField("12.0");
    private Controller controller;

    public InputPanel(Controller controller) {
        super(new GridLayout(1));
        trace.constructorCall();
        this.controller = controller;
        setBorder(MyBorderFactory.createMyBorder(" InputPanel "));
        add(new JLabel("Anzahl Punkte: "), new GridBagConstraints(0, 1, 1, 0.0, 0.0, GridBagConstraints.WEST,
            GridBagConstraints.NONE, new Insets(10, 10, 10, 0)));
        add(tfAnzahlPunkte, new GridBagConstraints(1, 0, 1, 1, 0.0, 0.0, GridBagConstraints.CENTER,
            GridBagConstraints.HORIZONTAL, new Insets(10, 10, 10, 0)));
        tfAnzahlPunkte.addActionListener(this);
        add(new JLabel("Maximal benötigte Punktzahl: "), new GridBagConstraints(0, 1, 1, 1, 0.0, 0.0,
            GridBagConstraints.WEST, GridBagConstraints.NONE, new Insets(10, 10, 10, 0)));
        add(tfMaxPunkte, new GridBagConstraints(1, 1, 1, 0.0, 0.0, GridBagConstraints.CENTER,
            GridBagConstraints.HORIZONTAL, new Insets(10, 10, 10, 0)));
        tfMaxPunkte.addActionListener(this);
        add(new JLabel("Note: "), new GridBagConstraints(0, 2, 1, 1, 0.0, 0.0, GridBagConstraints.WEST,
            GridBagConstraints.NONE, new Insets(10, 10, 10, 0)));
        add(tfNote, new GridBagConstraints(1, 2, 1, 1, 0.0, 0.0, GridBagConstraints.CENTER,
            GridBagConstraints.HORIZONTAL, new Insets(10, 10, 10, 0)));
        tfNote.setEditable(false);
        add(new JLabel(""), new GridBagConstraints(0, 3, 2, 1, 1.0, 1.0, GridBagConstraints.CENTER,
            GridBagConstraints.BOTH, new Insets(10, 10, 10, 0)));

        public void actionPerformed(ActionEvent e) {
            trace.eventCall();
            controller.btBerechne();
        }

        public void update(Observable obs, Object obj) {
            trace.methodCall();
            Model model = (Model) obs;
            double note = Math.round(10.0 * model.getNote()) / 10.0;
            tfNote.setText("" + note);
        }
    }
}
```

Controller

```
public class Controller {
    TraceVS trace = new TraceVS(this);
    private Model model;
    private static final int ENTER = 0, ZAHL = 1, OPERATION = 2;
    private int zustand = ENTER;

    public Controller(Model model) {
        trace.constructorCall();
        this.model = model;
    }

    public void add() {
        trace.methodCall();
        model.add();
        zustand = OPERATION;
    }
}
```

```

public void enter() {
    trace.methodCall();
    push();
}

public void setValue(int value) {
    trace.methodCall();
    stack[0] = value;
    notifyObservers();
}

public int getValue() {
    trace.methodCall();
    return stack[0];
}

private void pull() {
    trace.methodCall();
    for (int i = 1; i < stack.length - 1; i++) {
        stack[i] = stack[i + 1];
    }
    stack[stack.length - 1] = 0;
}

private void push() {
    trace.methodCall();
    for (int i = stack.length - 1; i > 0; i--) {
        stack[i] = stack[i - 1];
    }
    notifyObservers();
}

public String getStackInfo() {
    trace.methodCall();
    String s = "";
    for (int i = stack.length - 1; i >= 0; i--) {
        s += "" + stack[i] + "\n";
    }
    return s;
}

public void notifyObservers() {
    trace.methodCall();
    setChanged();
    super.notifyObservers();
}
}

```

```

public void subtract() {
    trace.methodCall();
    model.subtract();
    zustand = OPERATION;
}

public void multiply() {
    trace.methodCall();
    model.multiply();
    zustand = OPERATION;
}

public void divide() {
    trace.methodCall();
    model.divide();
    zustand = OPERATION;
}

public void enter() {
    trace.methodCall();
    model.enter();
    zustand = ENTER;
}

public void number(int i) {
    trace.methodCall();
    switch (zustand) {
        case ZAHL:
            model.setValue(model.getValue() * 10 + i);
            break;
        case OPERATION:
            model.enter();
            model.setValue(i);
            break;
        case ENTER:
            model.setValue(i);
            break;
    }
    zustand = ZAHL;
}
}

```

Model

```

public class Model extends Observable {
    TraceVS trace = new TraceVS(this);
    private int[] stack = new int[4];

    public Model() {
        trace.constructorCall();
    }

    public void add() {
        trace.methodCall();
        stack[0] = stack[0] + stack[1];
        pull();
        notifyObservers();
    }

    public void subtract() {
        trace.methodCall();
        stack[0] = stack[1] - stack[0];
        pull();
        notifyObservers();
    }

    public void divide() {
        trace.methodCall();
        stack[0] = stack[1] / stack[0];
        pull();
        notifyObservers();
    }

    public void multiply() {
        trace.methodCall();
        stack[0] = stack[0] * stack[1];
        pull();
        notifyObservers();
    }
}

```